

LSTM Architecture | Part 2 — “The How?”

1. The Big Picture: LSTM Architecture

The **LSTM (Long Short-Term Memory)** cell is a special type of RNN cell designed to handle **long-term dependencies** by introducing a **cell state** and **gates**.

An LSTM cell receives **three main inputs**:

- (x_t) : the input at the current time step
- (h_{t-1}) : the previous hidden state (short-term memory)
- (C_{t-1}) : the previous cell state (long-term memory)

And it produces **two outputs**:

- (h_t) : the current hidden state
 - (C_t) : the updated cell state
-

2. Internal Components of an LSTM Cell

Inside the cell, you’ll find:

1. **Forget Gate** → controls what to erase
2. **Input Gate** → controls what new info to add
3. **Cell State Updater** → updates the long-term memory
4. **Output Gate** → controls what to output at this time step

All of these components use **sigmoid (σ)** and **tanh** activations.

3. The Gates — Step-by-Step

Let’s see how each gate functions mathematically and intuitively.

(1) Forget Gate — “What should I forget?”

- **Formula:**

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- Inputs: previous hidden state and current input
- Output: a vector of values between 0 and 1
- If a value ≈ 0 → forget it
- If a value ≈ 1 → keep it

❖ **Intuition:** If the old info is no longer useful, the forget gate removes it from memory.

❖ **Example:**

“The movie was great, but the ending was bad.”

Forget early positive sentiment when processing “bad.”

(2) Input Gate — “What should I remember?”

Two steps happen here:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- i_t : decides which new info to add
- $\tilde{C}_t$: candidate values (potential new info)

The tanh layer creates new candidate memory content.

The sigmoid gate decides how much of it to actually store.

(3) Update Cell State — “Update the long-term memory”

Now the old memory (C_{t-1}) is updated into the new memory (C_t):

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

This combines:

- **Old memory (scaled by what to keep)**
- **New memory (scaled by what to add)**

Result: the new cell state (C_t) contains long-term info that’s useful going forward.

(4) Output Gate — “What do I output now?”

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

- The **output gate** decides what portion of the cell state becomes the visible output (h_t).
- (h_t) is also passed to the next time step.

Tanh squashes the cell state between -1 and 1.

Sigmoid in (o_t) filters how much to expose.

4. What Are C_t and H_t ?

Symbol	Meaning	Description
C_t	Cell State	Long-term memory that carries information across time steps
h_t	Hidden State	Short-term memory used for output and next-step input

They are interconnected — the **cell state** maintains continuity, while the **hidden state** reflects the current context.

5. What Is x_t ?

(x_t) is the **input vector** at time t .

It could be:

- A **word embedding** (in NLP tasks)
- A **feature vector** (in time series data)
- A **pixel sequence** (in video or handwriting data)

It's the information being fed into the model at that specific time step.

6. What Are f_t , i_t , o_t , and C_t ?

These are the outputs of the gates and memory at time t :

Symbol	Name	Function
(f_t)	Forget gate	How much old info to keep
(i_t)	Input gate	How much new info to add
(o_t)	Output gate	How much info to reveal
(C_t)	Cell state	The updated memory storage

Each is a **vector**, same size as the hidden state, controlling different aspects of memory flow.

7. Pointwise Operations

LSTMs use **element-wise (pointwise) multiplication**:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$h_t = o_t * \tanh(C_t)$$

Each element in these vectors interacts **independently**, allowing fine-grained control of memory flow — every neuron decides what to keep or forget individually.

8. Neural Network Layers Inside LSTM

Each gate (forget, input, output) is essentially a **tiny feedforward neural network**:

- Takes [h_{t-1} , x_t] as input
- Has its own weights (W) and biases (b)
- Uses **sigmoid/tanh** activations

So an LSTM cell actually contains **multiple dense layers internally**, all trained jointly.

9. The LSTM Cell (Full Architecture)

Let's summarize all the steps together visually (conceptually):

Inputs: x_t, h_{t-1}, C_{t-1}

- 1 Forget gate: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
- 2 Input gate: $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
- 3 Candidate: $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$
- 4 Update cell: $C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$
- 5 Output gate: $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$
- 6 Hidden state: $h_t = o_t \cdot \tanh(C_t)$

Outputs: h_t, C_t

10. Key Insights

Concept	Explanation
Parallel info flow	Cell state carries long-term info; hidden state carries immediate info
Gradient stability	The additive update of (C_t) prevents gradient vanishing
Multiple gates	Allow selective memory control
Trainable layers	Each gate has its own weights & biases
Pointwise ops	Enable neuron-level control over information retention

11. In a Nutshell

The LSTM cell is like a *mini computer* that decides what to remember, forget, and output at each time step.

It keeps **long-term context** through (C_t), while dynamically adjusting what's relevant through the gates — making it far more robust than a vanilla RNN.

➤ Summary Table

Symbol	Meaning	Activation	Purpose
(f_t)	Forget gate	Sigmoid	Remove irrelevant info
(i_t)	Input gate	Sigmoid	Select important new info
($\tilde{C}_t$)	Candidate memory	Tanh	New memory candidate
(C_t)	Cell state	—	Long-term memory
(o_t)	Output gate	Sigmoid	Control visible output
(h_t)	Hidden state	Tanh	Short-term context output